

A decorative background on the left side of the page featuring a red circuit board pattern with various traces and components.

Tecnológico Nacional de México

Instituto Tecnológico de Chiná

Nombre de la Carrera:

INGENIERÍA INFORMÁTICA

Nombre de la Asignatura:

PROGRAMACION CLIENTE SERVIDOR

Semestre y grupo:

7AI

Actividad:

INVESTIGACIÓN

Nombre del estudiante:

MIGUEL EDUARDO CHI CHABLE

Nombre del Profesor:

ING.JENNER NOEL CHE MÉNDEZ

Lugar y Fecha:

CHINÁ, CAMPECHE, 04 OCTUBRE DEL 2026

INDICE

Tabla de contenido

1.1 Evolución del modelo cliente servidor 4

1.2 Fundamentos de red aplicados 6

1.3 Arquitecturas de dos tres y N capas 8

1.4 Estilos y protocolos de comunicación 10

1.5 Criterios de calidad y sostenibilidad 12

1.6 Diseño API first C4 contrato inicial y ADR..... 13

Conclusión..... 15

Bibliografía..... 16

Introducción

Las aplicaciones conectadas a una red necesitan mecanismos para intercambiar información, distribuir tareas y ofrecer servicios confiables. El modelo cliente-servidor constituye una base para comprender este funcionamiento: un proceso solicita una operación y otro la atiende. A partir de esta relación se desarrollan arquitecturas que organizan la presentación, la lógica de negocio, el almacenamiento y la comunicación.

Esta investigación aborda la evolución del modelo cliente-servidor, los fundamentos de red, las principales arquitecturas y los mecanismos de comunicación. También explica los criterios de calidad y las prácticas de documentación que ayudan a diseñar sistemas mantenibles.

1.1 Evolución del modelo cliente servidor

El modelo cliente-servidor distribuye el trabajo entre procesos que solicitan servicios y procesos que los proporcionan. Los términos cliente y servidor describen funciones dentro de una interacción; un mismo programa puede actuar como servidor al recibir una solicitud y como cliente al consultar otro servicio.

Proceso cliente

Es un programa en ejecución que inicia una solicitud para obtener información o realizar una operación. Sus responsabilidades pueden incluir presentar la interfaz, recopilar datos del usuario, enviar peticiones y procesar respuestas. El cliente debe considerar que la comunicación puede fallar o tardar, por lo que necesita manejar errores y tiempos de espera.

Proceso servidor

Es un programa que recibe solicitudes, ejecuta operaciones y devuelve resultados. Puede validar datos, comprobar permisos, aplicar reglas de negocio y acceder a bases de datos. Un servidor puede atender múltiples clientes mediante distintos mecanismos de concurrencia.

El servidor no necesariamente corresponde a una computadora física exclusiva: puede ejecutarse en una máquina virtual, un contenedor o una plataforma administrada.

Servicio

Es una capacidad que un sistema ofrece mediante una interfaz definida. El servicio establece qué operaciones están disponibles, qué información requieren y qué resultados producen. Su contrato permite que los clientes lo utilicen sin conocer todos los detalles de su implementación.

Middleware

El middleware es software que facilita la comunicación o integración entre aplicaciones y componentes. Puede proporcionar mensajería, transformación de datos, autenticación, gestión de conexiones o coordinación de operaciones. Ayuda a conectar sistemas que utilizan tecnologías diferentes.

Sistema distribuido

Es un conjunto de componentes que se ejecutan en distintos nodos y colaboran mediante una red. La distribución permite repartir trabajo y recursos, pero introduce dificultades: retrasos de comunicación, fallos parciales, coordinación y sincronización de datos.

A diferencia de una llamada dentro de un mismo proceso, una operación remota puede fallar aunque el programa que la solicita continúe funcionando.

Evolución del modelo

La evolución puede entenderse como una ampliación de la distribución de responsabilidades:

1. **Computación centralizada:** el procesamiento se concentra en un sistema central y las terminales ofrecen acceso.
2. **Cliente-servidor de dos niveles:** el cliente realiza parte del procesamiento y se comunica directamente con el servidor, frecuentemente de base de datos.
3. **Tres niveles:** se incorpora un servidor de aplicaciones que separa la interfaz del acceso a datos.
4. **Sistemas distribuidos:** múltiples servicios colaboran y se despliegan en distintos nodos.
5. **Servicios administrados en la nube:** parte de la infraestructura y de las funciones del backend se delega al proveedor.

1.2 Fundamentos de red aplicados

Los fundamentos de red explican cómo una solicitud encuentra su destino, llega a un servicio y obtiene una respuesta.

Concepto	Definición y función
DNS	Sistema de nombres de dominio. Permite consultar registros asociados con nombres; entre ellos, los que relacionan un dominio con direcciones IP.
IP	Protocolo que permite direccionar y transportar paquetes entre redes. Una dirección IP identifica una interfaz dentro de un contexto de red.
Puertos	Identificadores numéricos utilizados por protocolos de transporte para distinguir puntos de comunicación. Permiten que una máquina atienda distintos servicios.
TCP	Protocolo de transporte orientado a conexión que entrega un flujo de bytes ordenado y utiliza retransmisiones ante pérdidas.
HTTP	Protocolo de aplicación que define cómo intercambiar peticiones y respuestas, incluyendo métodos, encabezados y códigos de estado.
HTTPS	Comunicación HTTP protegida mediante TLS.
Petición	Mensaje mediante el cual el cliente solicita una operación. Puede incluir método, destino, encabezados y cuerpo.
Respuesta	Mensaje que devuelve el servidor. Incluye un código de estado, encabezados y, cuando corresponde, un cuerpo.
Latencia	Tiempo de demora en una comunicación. Debe distinguirse entre latencia de red y tiempo total de respuesta, que también incluye procesamiento.
TLS	Protocolo que protege la comunicación mediante cifrado, integridad y autenticación, normalmente del servidor mediante certificados.

DNS permite localizar destinos por nombre, mientras que TCP establece la conexión y proporciona transporte confiable. HTTP define el intercambio de mensajes; HTTPS añade protección criptográfica.

Funcionamiento conjunto

En una conexión HTTPS nueva sobre TCP, el cliente puede consultar DNS, establecer la conexión TCP y negociar TLS antes de intercambiar mensajes HTTP. Algunos pasos se evitan o reducen cuando existen resultados en caché o conexiones reutilizadas.

Los puertos habituales son 80 para HTTP y 443 para HTTPS, aunque pueden configurarse otros. HTTP/1.1 y HTTP/2 suelen utilizar TCP; HTTP/3 utiliza QUIC sobre UDP. Por ello, HTTP no debe considerarse inseparable de TCP.

TLS protege los datos durante su transmisión, pero la aplicación sigue siendo responsable de autorizar operaciones, validar entradas y proteger la información almacenada.

1.3 Arquitecturas de dos tres y N capas

Una arquitectura define cómo se organizan los componentes y cómo colaboran. Es necesario distinguir entre capas lógicas, que separan responsabilidades en el software, y niveles físicos, que separan unidades de despliegue. Varias capas pueden ejecutarse dentro de una misma aplicación.

Arquitectura de dos capas

Se organiza habitualmente en un cliente y un servidor de datos. El cliente contiene la presentación y parte de la lógica, y accede directamente al servidor.

Su ventaja es una estructura sencilla. Sus dificultades aparecen cuando deben distribuirse actualizaciones a muchos clientes o controlar el acceso directo a los datos.

Arquitectura de tres capas

Separa tres responsabilidades:

- Presentación: interacción con el usuario.
- Lógica de negocio: reglas, validaciones y operaciones.
- Datos: almacenamiento y recuperación de información.

Esta organización facilita modificar una responsabilidad sin intervenir necesariamente en las demás.

Arquitectura de N capas

Amplía la separación mediante capas adicionales, como integración o acceso a datos. Puede mejorar la organización, pero demasiadas capas añaden dependencias y recorridos innecesarios. Cada capa debe tener una responsabilidad justificada.

Monolito modular

Es una aplicación desplegada como una unidad, organizada internamente en módulos con límites claros. Facilita el despliegue y la ejecución de operaciones dentro de un mismo proceso. Su principal reto es conservar la independencia entre módulos y evitar dependencias desordenadas. La modularidad permite mantener una aplicación monolítica bien estructurada.

Microservicios

Dividen el sistema en servicios que representan capacidades del negocio y pueden desplegarse de forma independiente. Permiten escalar o actualizar partes específicas, pero requieren resolver comunicación remota, observabilidad, fallos parciales y gestión de datos distribuidos.

Arquitectura orientada a eventos

Los componentes publican eventos que describen hechos ocurridos y otros componentes reaccionan a ellos. Puede utilizar publicación-suscripción o flujos de eventos.

Reduce dependencias temporales entre productores y consumidores. Sus retos incluyen duplicados, orden de procesamiento y seguimiento de operaciones que atraviesan varios componentes.

Serverless

Es un modelo en el que el proveedor administra servidores y recursos de ejecución. El equipo se concentra en el código y su configuración. Incluye funciones bajo demanda y otros servicios administrados.

Puede reducir tareas de infraestructura, aunque deben evaluarse límites de ejecución, costos variables y dependencia del proveedor.

Backend as a Service o BaaS

Ofrece capacidades de backend listas para integrar mediante API o SDK, como autenticación, bases de datos y almacenamiento. Reduce el trabajo de implementar funciones comunes, pero requiere revisar permisos, costos y posibilidades de migración. Serverless y BaaS pueden combinarse. [11]

Estas opciones describen aspectos diferentes: las capas organizan responsabilidades, los microservicios organizan unidades de servicio, los eventos organizan interacciones y serverless describe la administración de la ejecución.

1.4 Estilos y protocolos de comunicación

REST

REST es un estilo arquitectónico, definido mediante restricciones como cliente-servidor, comunicación sin estado, caché, interfaz uniforme y sistema por capas; el código bajo demanda es opcional.

En APIs HTTP, suele representar recursos mediante URI y utilizar métodos como GET, POST, PUT o DELETE. Una API no cumple REST únicamente por utilizar HTTP y JSON. La condición “sin estado” significa que cada solicitud contiene la información necesaria para atenderla, sin depender de un contexto de sesión conservado en el servidor entre solicitudes.

RPC y gRPC

RPC permite solicitar la ejecución de una operación remota mediante una interfaz de procedimientos. Aunque su uso pueda parecer una llamada local, conserva los riesgos de comunicación de una red.

gRPC es un framework de RPC que suele utilizar Protocol Buffers para definir contratos y serializar mensajes. Permite generar código para clientes y servidores y admite llamadas individuales y distintas modalidades de streaming.

WebSocket

Establece un canal persistente y bidireccional. Cliente y servidor pueden enviar mensajes sin iniciar una petición HTTP nueva por cada intercambio. Requiere gestionar conexiones, reconexiones y velocidad de procesamiento de mensajes.

Server Sent Events o SSE

Permite que el servidor envíe eventos de texto al cliente mediante una conexión HTTP mantenida abierta. Es un mecanismo unidireccional: los mensajes del cliente hacia el servidor requieren otro canal, como peticiones HTTP.

Webhooks

Son notificaciones que un sistema envía a una URL registrada cuando ocurre un evento. Normalmente se entregan mediante peticiones HTTP. Evitan consultar constantemente al emisor para detectar cambios. El receptor debe comprobar la autenticidad y manejar posibles entregas repetidas.

Criterios de selección

La siguiente comparación sintetiza los mecanismos anteriores:

Mecanismo	Cuándo considerarlo	Aspecto que debe evaluarse
REST sobre HTTP	Operaciones sobre recursos y amplia interoperabilidad.	Semántica HTTP, versiones, caché y errores.
RPC/gRPC	Operaciones remotas con contratos definidos y comunicación entre servicios.	Compatibilidad de clientes, herramientas y evolución del contrato.
WebSocket	Intercambio continuo en ambas direcciones.	Conexiones simultáneas, reconexión y control de carga.
SSE	Actualizaciones continuas del servidor al cliente.	Dirección del flujo y duración de las conexiones.
Webhooks	Notificaciones entre sistemas independientes.	Autenticidad, reintentos e idempotencia.
Mensajería	Procesamiento asíncrono y desacoplamiento temporal.	Durabilidad, orden, duplicados y supervisión.

La elección debe considerar dirección de la comunicación, frecuencia, volumen, latencia esperada y capacidad operativa del equipo.

1.5 Criterios de calidad y sostenibilidad

Separación de responsabilidades

Consiste en asignar funciones claras a módulos, capas o servicios. Busca alta cohesión, es decir, que las funciones relacionadas permanezcan juntas, y bajo acoplamiento, para reducir dependencias innecesarias.

Esta separación facilita el mantenimiento y las pruebas. Puede lograrse dentro de un monolito; no exige distribuir el sistema.

Escalabilidad

Es la capacidad de atender un crecimiento de carga conservando niveles aceptables de servicio.

Vertical: aumentar recursos de una instancia.

Horizontal: añadir instancias y distribuir el trabajo.

La escalabilidad depende también de bases de datos, almacenamiento, coordinación y servicios externos. Agregar servidores no elimina automáticamente los cuellos de botella.

Disponibilidad

Es la capacidad del sistema para prestar servicio cuando se necesita. Se favorece mediante redundancia, supervisión, recuperación automática y eliminación de puntos únicos de fallo.

Debe medirse desde la experiencia del usuario: una infraestructura activa puede coexistir con una aplicación que no permite completar operaciones.

Costo

Incluye infraestructura, almacenamiento, transferencia, licencias, desarrollo y operación. Una solución debe evaluarse por su costo total, considerando tanto la demanda habitual como los picos y el esfuerzo de mantenerla.

Complejidad

Abarca la dificultad para comprender, desarrollar, desplegar y diagnosticar el sistema. Distribuir componentes puede facilitar su evolución independiente, pero añade comunicación remota y coordinación. Es necesario que los beneficios compensen estas dificultades.

Deuda operativa

Es la acumulación de carencias que dificultan operar el sistema: despliegues manuales, alertas insuficientes, dependencias desactualizadas, procedimientos incompletos o recuperación no verificada.

1.6 Diseño API first C4 contrato inicial y ADR

Diseño API first

Es un enfoque que define y revisa la interfaz de comunicación antes de desarrollar su implementación. Permite acordar cómo colaborarán los consumidores y proveedores de una API.

No significa diseñar únicamente rutas: también implica establecer datos, errores, seguridad y reglas de compatibilidad. OpenAPI ofrece un formato estándar para describir APIs HTTP que puede servir como contrato compartido.

Contrato inicial

Es la primera especificación acordada de la interfaz. Debe incluir:

- Propósito y alcance.
- Operaciones disponibles.
- Parámetros y campos obligatorios.
- Estructuras de petición y respuesta.
- Códigos y formatos de error.
- Autenticación y permisos.
- Reglas de paginación, límites e idempotencia cuando correspondan.
- Política de versiones y compatibilidad.

El contrato permite revisar la integración antes de completar el sistema, construir simulaciones y verificar posteriormente que la implementación cumple lo acordado.

Diagramas C4 simplificados

C4 describe la estructura del software mediante niveles de detalle:

Nivel	Información que representa
Contexto	El sistema, sus usuarios y los sistemas externos relacionados.
Contenedores	Aplicaciones y almacenes de datos, sus responsabilidades y comunicaciones.
Componentes	Partes principales dentro de un contenedor.
Código	Estructuras detalladas de implementación.

En C4, un contenedor es una aplicación o un almacén de datos; no necesariamente es un contenedor Docker. Para muchos equipos, los diagramas de contexto y contenedores resultan suficientes. Los niveles adicionales se elaboran cuando aportan información útil.

Una versión simplificada debe mantener límites claros, nombres comprensibles, responsabilidades y relaciones etiquetadas.

Registros de decisión arquitectónica o ADR

Un ADR documenta una decisión relevante, el contexto que la motivó y sus consecuencias. Conserva el razonamiento para que futuros integrantes comprendan por qué se eligió una solución.

Su estructura habitual incluye:

1. Título e identificador.
2. Estado: propuesto, aceptado o reemplazado.
3. Contexto: problema, requisitos y restricciones.
4. Alternativas: opciones consideradas.
5. Decisión: solución elegida y razones.
6. Consecuencias: beneficios, costos y limitaciones.

Cuando una decisión cambia, conviene conservar el registro anterior y relacionarlo con el nuevo. Así se mantiene la historia del diseño.

Conclusión

El diseño de sistemas cliente-servidor requiere comprender tanto la comunicación de red como la organización del software. Cada arquitectura y mecanismo de intercambio ofrece beneficios acompañados de costos y dificultades. Su selección debe responder a requisitos concretos de rendimiento, disponibilidad, consistencia y mantenimiento.

La separación de responsabilidades, los contratos de API, los diagramas C4 y los ADR ayudan a construir sistemas comprensibles y a justificar sus decisiones. Una arquitectura adecuada permite satisfacer las necesidades actuales y evolucionar con un esfuerzo operativo sostenible.

Bibliografía

(S/f). Apinem.com. Recuperado el 5 de octubre de 2026, de <https://www.apinem.com/el-modelo-cliente-servidor/>

Arquitectura cliente servidor: qué es, tipos y ejemplos. (s/f). Arsys. Recuperado el 5 de octubre de 2026, de <https://www.arsys.es/blog/todo-sobre-la-arquitectura-cliente-servidor>

1.1 El modelo cliente-servidor tradicional. (s/f). Campusempresa.com. Recuperado el 5 de octubre de 2026, de <https://campusempresa.com/cursos/cloud-aws-terraform/01-01-modelo-cliente-servidor>

Redes de Computadoras - 1.2 Conceptos básicos y componentes de una red. (s/f). Google.com. Recuperado el 5 de octubre de 2026, de <https://sites.google.com/tectijuana.mx/redes-de-computadoras-1/unidad-1introducci%C3%B3n-a-las-redes-de-datos/1-2-conceptos-b%C3%A1sicos-y-componentes-de-una-red>

DFDR_ Unidad 1. (s/f). Unadmexico.mx. Recuperado el 5 de octubre de 2026, de <https://dmd.unadmexico.mx/contenidos/DCEIT/BLOQUE2/DS/05/DFDR/U1/Unidad1.html>

Client challenge. (s/f). Scribd.com. Recuperado el 5 de octubre de 2026, de <https://es.scribd.com/document/433388294/1-2-Modelo-de-Dos-y-Tres-Capas>

Por, R. (2013, marzo 8). *sistemasdistribuidostnet*. sistemasdistribuidostnet. <https://sistemasdistribuidosdotnet.wordpress.com/2013/03/08/1-1-3-aplicaciones-de-23-y-n-capas/>

Manual del Sistema de Gestión Ambiental. (s/f). Tecnm.mx. Recuperado el 5 de octubre de 2026, de https://www.tecnm.mx/dir_calidad/gestion_ambiental/pdf/TecNM-GA-MA.pdf

Requisitos de la norma ISO 9001:2015, cláusula por cláusula. (2025, enero 10). Softgrade. <https://softgrade.mx/requisitos-iso-9001/>